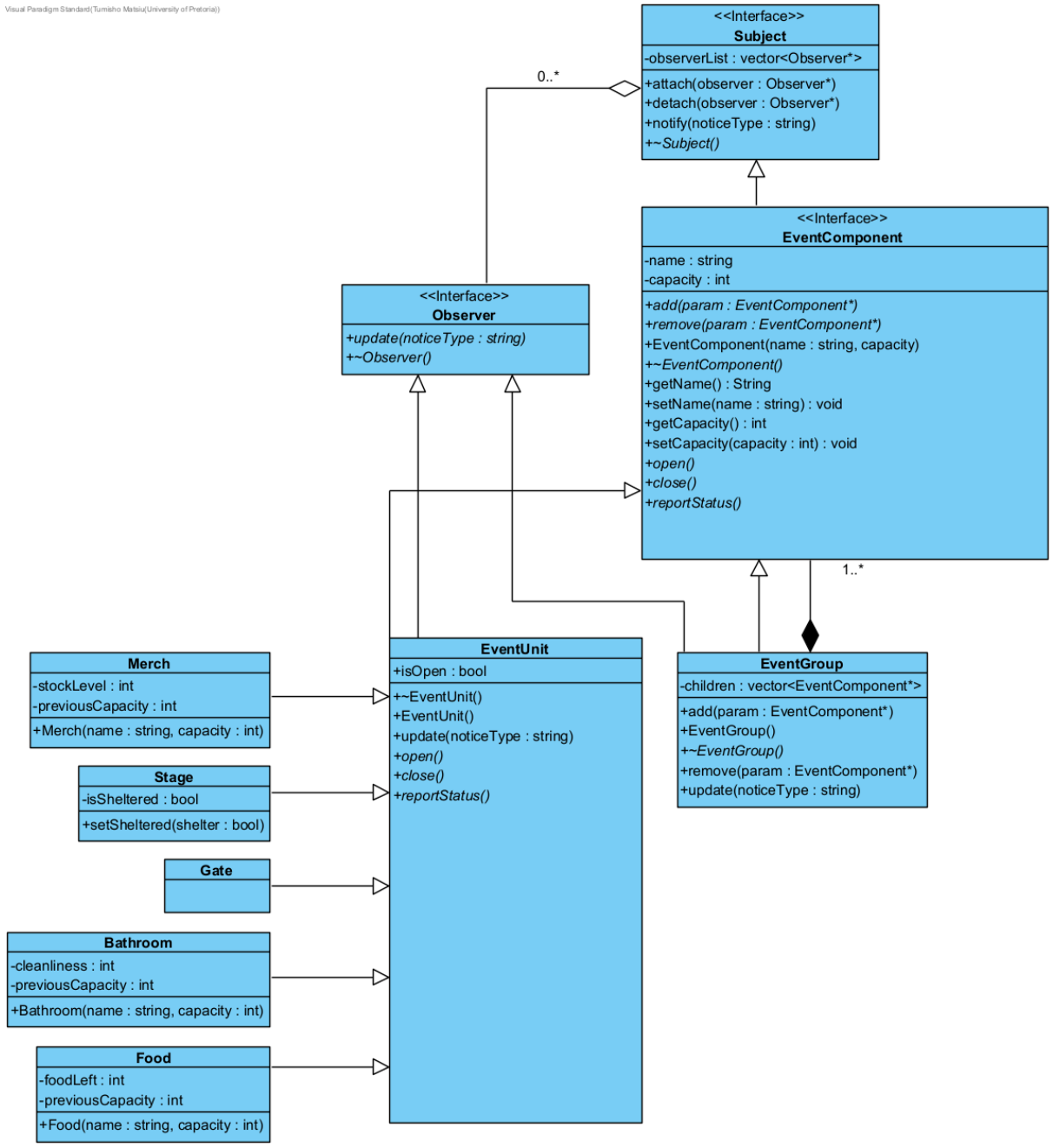
1.1
Our event is named “Spring Day” . It is a music festival with different stages that will each play a different genre of music and sell food and merchandise associated with that genre. Our five concrete event units are going to be a merchandise vendor, a food and drink vendor, a bathroom, a stage and a gate. Our grouping areas are going to be the main event group and then there will be a house music area, a rock music area and a pop music area that could each have their own vendors, stages, bathrooms and a gate.

1.2
Composite Design Pattern:
**Component:** EventComponent
**Leaf:** EventUnit, Merch, Food, Bathroom, Stage, Gate
**Composite:** EventGroup

Observer Design Pattern:
**Observer:** Observer
**ConcreteObserver:** EventUnit (and its leaves), EventGroup
**Subject:** Subject
**ConcreteSubject:** EventGroup and the derived classes of EventUnit

EventUnit and EventGroup are both participants in Observer and Composite design patterns. EventUnit and EventGroup both interface from EventComponent which inherits from Subject making them behave as subjects allowing them to attach, detach and notify operations. The leaves of EventUnit and EventGroup would act as concreteObservers. They need to be able to observe other EventUnits or EventGroups for status or capacity changes.

1.3




1.4

a)

The fact that an EventGroup can have another EventGroup as a child node, which could have its own leaves or more event groups is what makes this a real part-whole tree.

b) Using the Observer pattern allows us to pass down information from EventComponent to EventComponent without modifying the code of the observers every time the subject changes.

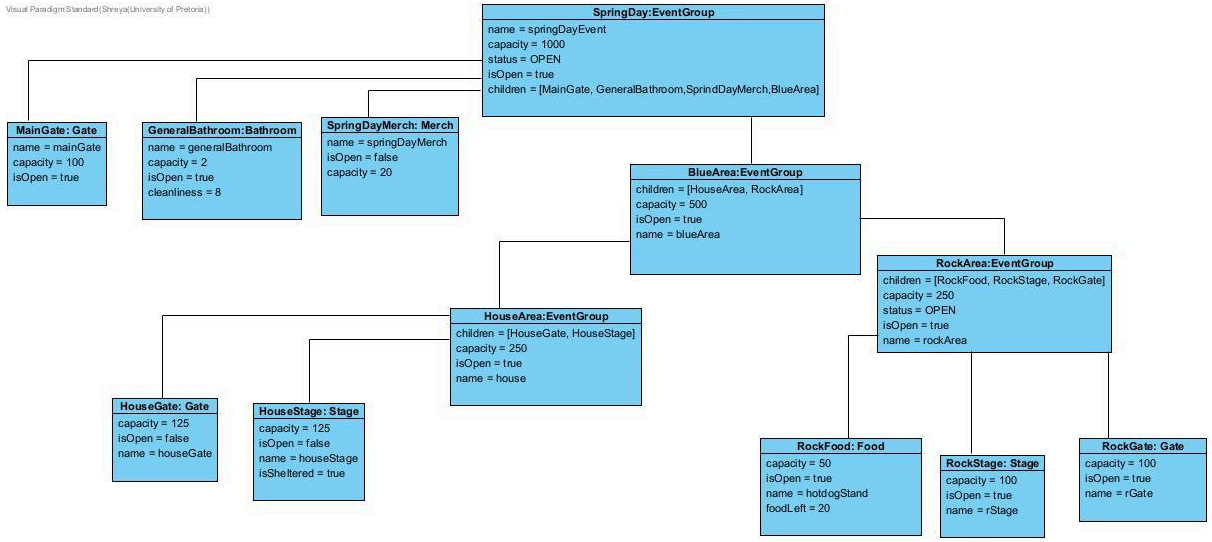
c) An EventGroup owns its children, which could be other EventGroups or EventUnits and is thus responsible for deallocating them.

d) The subject, EventComponent, EventGroup and all the leaves hold observer references because they all inherit from Subject. It is only the observer that does not.

e) It is not a misuse as the subject itself can be an observer that observes another subject. The operations of a subject are interdependent from the operations of an observer. Observers are focused on changes in state of the subject while the subject's role is to provide these updates to the observer.

## **Task 2**

2.3




2.4 The destruction of the root, will call its destructor which will recursively destruct all its children, which will each destruct their own children and so on.

## **Task 3:**

3.3
Notice Types:
OPEN (operational change)
CLOSE (operational change)
CAPACITY_ALERT (capacity change):
WEATHER_ALERT (operational change/safety-related change)
EVACUATE (safety related change)
GIVEAWAY (operational change)

3.4 tested in main

3.5
We decided on implementing a push Observer. A major trade off is that the notify operation has to iterate through every registered observer every time the subject's state changes which will slow down our programme as the system and provide data to observers that do not need this new data but it reduces the complexity of having to query the subject every time when implementing the pull variation.  The notify function takes in a NoticeType string and passes that down into the update function. Each observer will determine how it will react to the notice type within their respective implementations of the update function.

## **Task 4**

4.1
weather alert:

- a food stand will  close
- a stage may close if it is not sheltered,
- a Gate will stay open
- a bathrooms cleanliness will decrease
- a merch store will close.

Capacity alert:

- If capacity > 1500 for an EventGroup then it closes all its gates
- If capacity increments from previous value then a Merch store's stock levels go down (same applies to Food levels for a Food unit)
- If capacity incremented from the previous values decrease cleanliness of bathroom and if it decremented increase cleanliness of bathroom
- Stage closes if capacity is >= 1000

Giveaway:

- If a Merch Unit has a stockLevel greater than 200, decrement it by 20
- If food Unit capacity is low (less than 100) decrement foodLeft by
- All the other leaves remain the same

Evacuate:

- All gates must be made be opened
- All other units should be closed
- Merch and Food levels should be set to 0

Open:

- Set EventUnit/EventGroup to open (change flag)

Close:

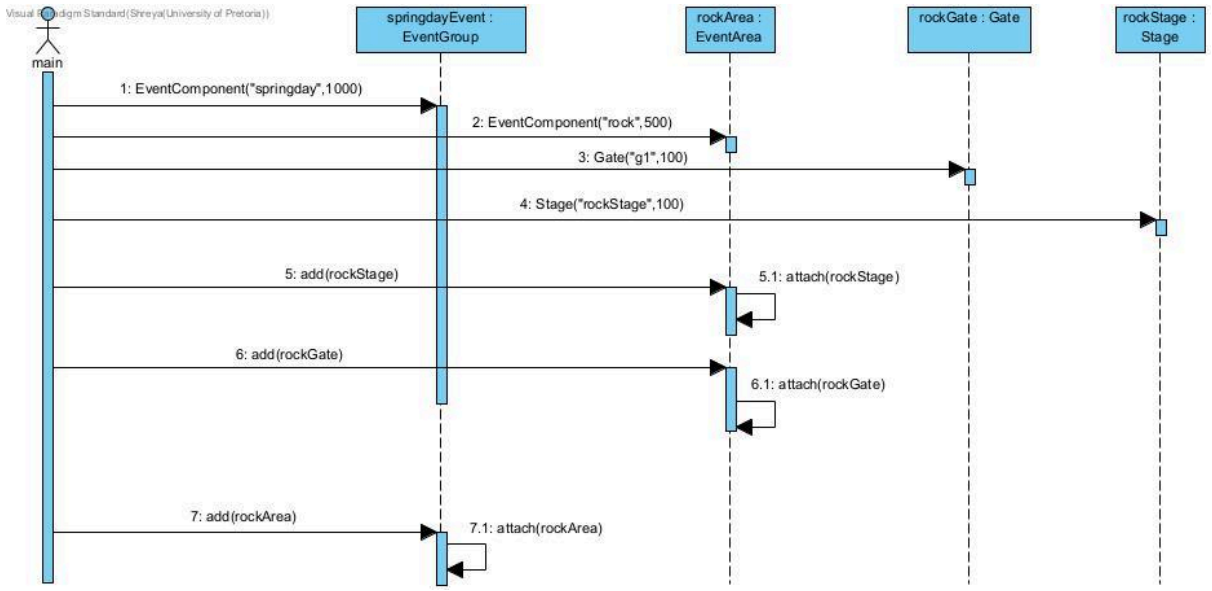
- Set EventUnit/EventGroup to closed (change flag)

4.4

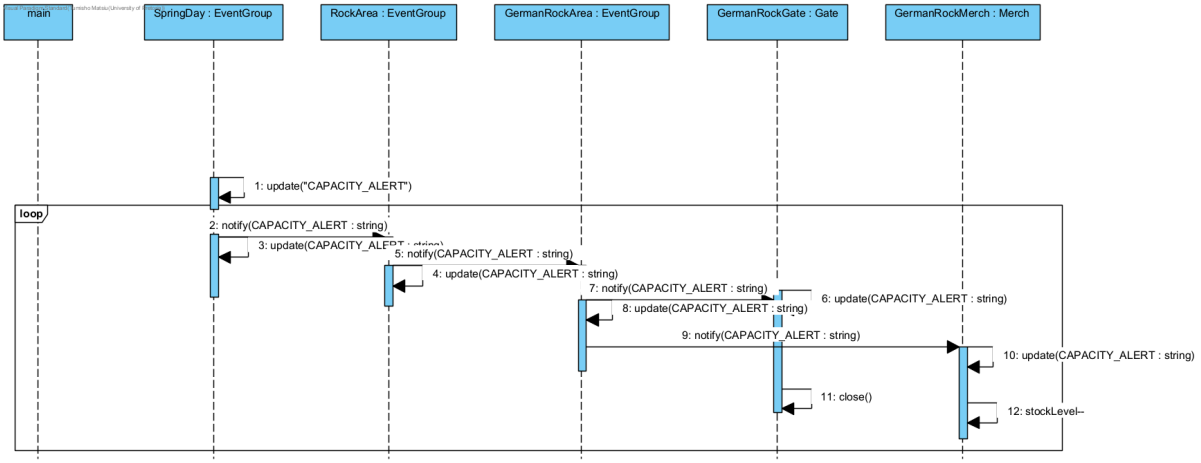
We added extra relevant attributes and operations to the leaves of the composite so that they respond differently to updates. The Stage unit can be sheltered or not  allowing it to behave differently depending on the weather alert and is specific to the Stage unit. Merch and Food Units have product levels affecting how they behave with giveaway and capacity alert events.
Bathrooms have a cleanliness indicator that is affected by the weather and capacity events.

## **Task 5**

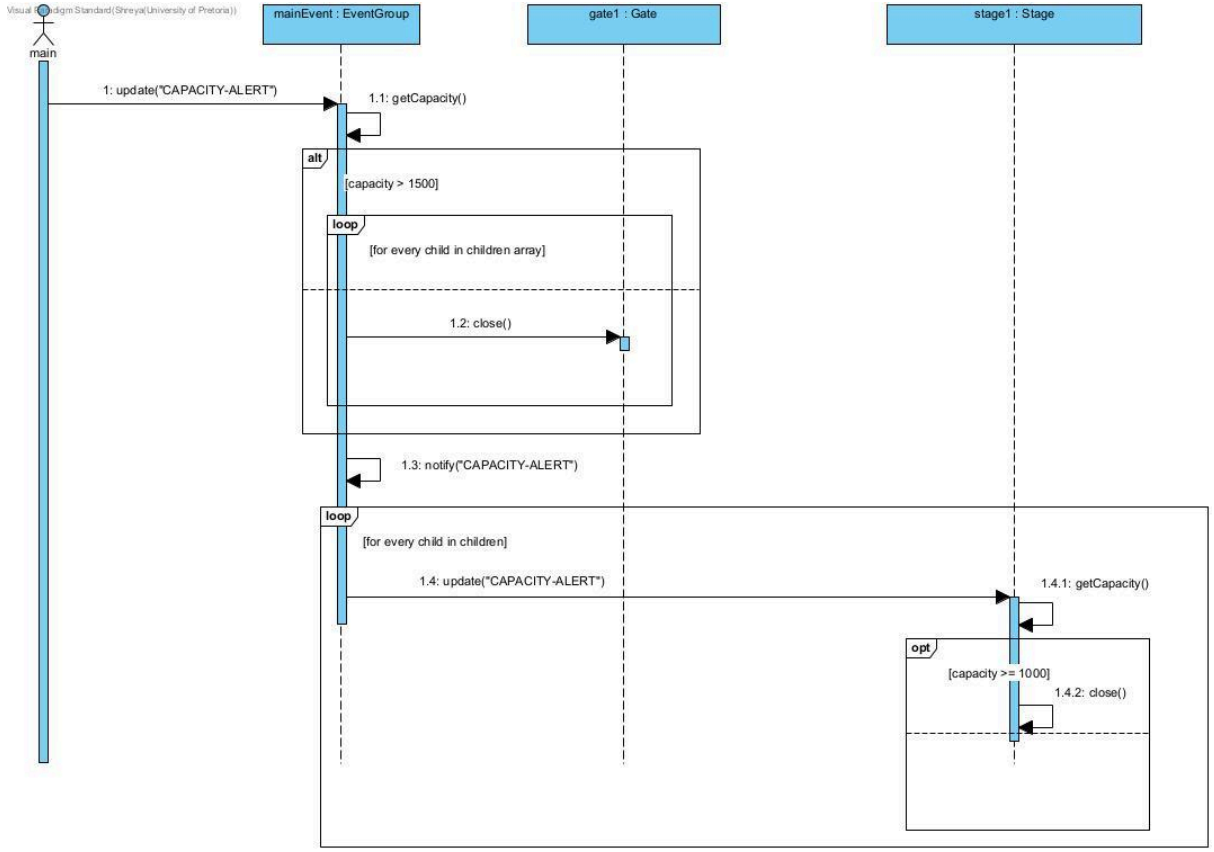
SD1



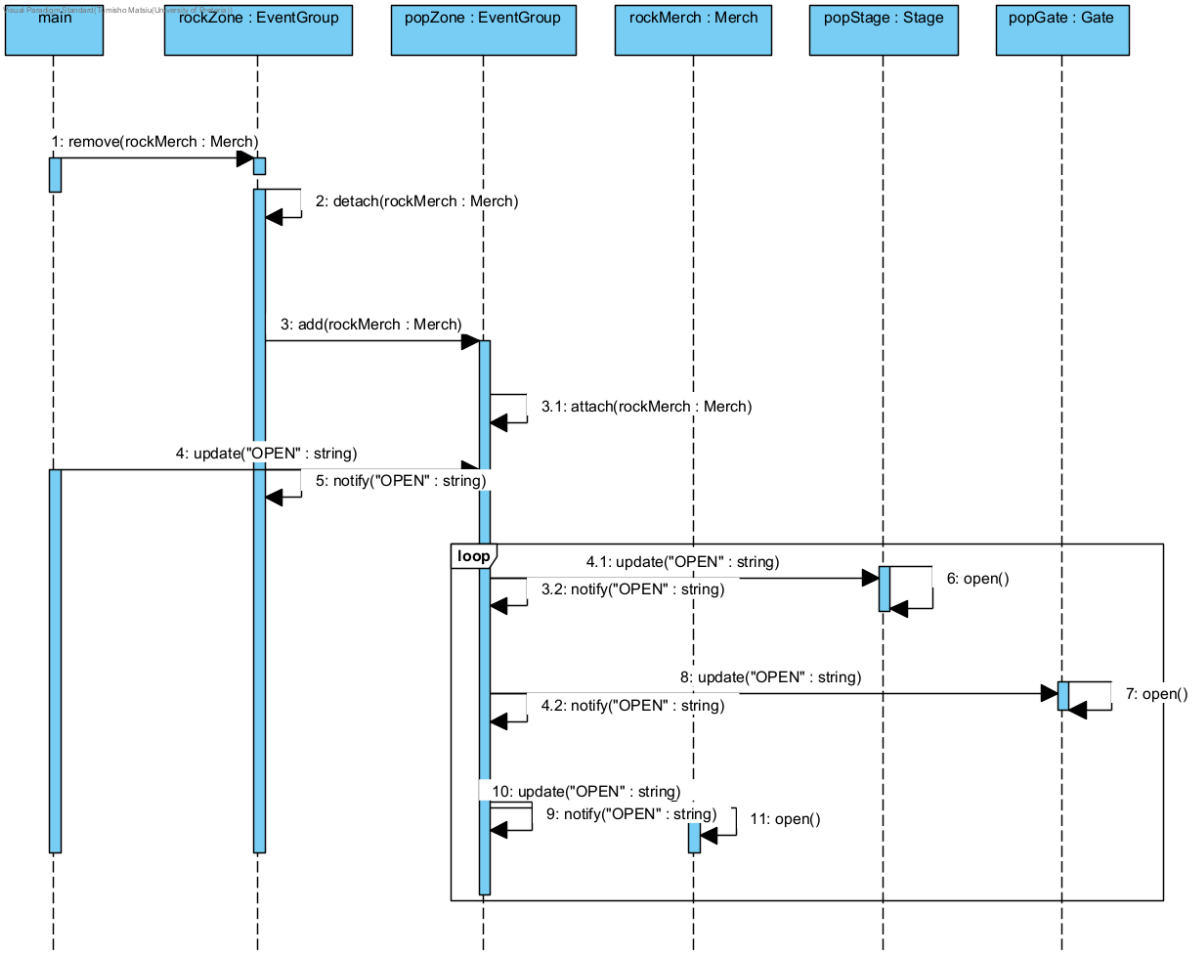
SD2


main
SpringDay : EventGroup
RockArea : EventGroup
GermanRockArea : EventGroup
GermanRockGate : Gate
GermanRockMerch : Merch
1: update("CAPACITY_ALERT")
loop
2: notify(CAPACITY_ALERT : string)
3: update(CAPACITY_ALERT : string)
5: notify(CAPACITY_ALERT : string)
4: update(CAPACITY_ALERT : string)
7: notify(CAPACITY_ALERT : string)
6: update(CAPACITY_ALERT : string)
8: update(CAPACITY_ALERT : string)
9: notify(CAPACITY_ALERT : string)
10: update(CAPACITY_ALERT : string)
11: close()
12: stockLevel--

SD3


main
mainEvent : EventGroup
gate1 : Gate
stage1 : Stage
1: update("CAPACITY-ALERT")
1.1: getCapacity()
alt
[capacity > 1500]
loop
[for every child in children array]
1.2: close()
1.3: notify("CAPACITY-ALERT")
loop
[for every child in children]
1.4: update("CAPACITY-ALERT")
1.4.1: getCapacity()
opt
[capacity >= 1000]
1.4.2: close()

SD4




**Task 7:**

We divided work in terms of theory questions and program planning/ structure, Composite Design Pattern and Observer Pattern. These roles would swap between teammates depending on their knowledge of the current topic, allowing us to effectively collaborate and showcase our individual strengths as a team. We created separate branches for each teammate to work on, allowing us to independently collaborate and pull from each other's branches before pushing to the main branch.

We strictly ensure before making any merges or commits to the main branch, we discuss the changes we are making with each and approve or disapprove. We also strictly ensured that meaningful comments were assigned to our merge and commit messages. A meaningful commit would be the process of getting towards our final UML diagram, each commit showing the iterative changes made and reaching a final agreed upon system design. The reportStatus function commit in the concrete Leaves shows meaningful collaboration as the multiple commits show the many changes we made in discussion that would align with the whole team's goals. A final commit would be the initial headers file commit, setting a skeleton for the team to work off and build upon.